

Сувенири

Леонид купува сувенири во една странска продавница. Постојат N **типови** на сувенири. Во продавницата има бесконечно многу сувенири од секој тип.

Секој тип на сувенир има фиксна цена. Имено, сувенир од типот i ($0 \leq i < N$) има цена од $P[i]$ монети, каде што $P[i]$ е позитивен цел број.

Леонид знае дека типовите на сувенири се сортирани во опаѓачки редослед според цената, и дека цените на сувенирите се различни. Поточно, $P[0] > P[1] > \dots > P[N-1] > 0$. Уште повеќе, тој успеал да ја дознае вредноста на $P[0]$. За жал, Леонид нема други информации за цените на сувенирите.

За да купи сувенири, Леонид ќе изврши одреден број трансакции со продавачот.

Секоја трансакција се состои од следниве чекори:

1. Леонид му подава одреден (позитивен) број монети на продавачот.
2. Продавачот ги става овие монети во купче на масата во задната соба, каде што Леонид не може да ги види.
3. Продавачот ги разгледува сите сувенири од типот $0, 1, \dots, N-1$ по тој редослед, еден по еден. Секој тип се разгледува **точно еднаш** по трансакција.
 - Кога се разгледува сувенир од типот i , ако моменталниот број на монети во купчето е најмалку $P[i]$, тогаш
 - продавачот отстранува $P[i]$ монети од купчето, и
 - продавачот става еден сувенир од типот i на масата.
4. Продавачот му ги дава на Леонид сите монети што останале во купчето и сите сувенири на масата.

Да забележиме дека на масата нема монети или сувенири пред да започне трансакција.

Ваша задача е да му наложите на Леонид да изврши одреден број трансакции, така што:

- во секоја трансакција тој купува **барем еден** сувенир, и
- вкупно тој купува **точно** i сувенири од типот i , за секое i такво што $0 \leq i < N$. Имајте предвид дека ова значи дека Леонид не треба да купува ниту еден сувенир од типот 0.

Леонид не мора да го минимизира бројот на трансакции и има неограничена количина на монети.

Имплементациски детали

Треба да ја имплементирате следната процедура.

```
void buy_souvenirs(int N, long long P0)
```

- N : бројот на типови на сувенири.
- $P0$: вредноста на $P[0]$.
- Оваа процедура се повикува точно еднаш за секој тест случај.

Горната процедура може да прави повици до следната процедура, за да му наложи на Леонид да изврши трансакција:

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- M : бројот на монети што ги подава Леонид на продавачот.
- Процедурата враќа пар. Првиот елемент на овој пар е низа L , што ги содржи типовите на сувенири што се купени (во растечки редослед). Вториот елемент е цел број R , бројот на монети вратени на Леонид по трансакцијата.
- Потребно е $P[0] > M \geq P[N - 1]$. Условот $P[0] > M$ гарантира дека Леонид нема да купи никаков сувенир од типот 0, а $M \geq P[N - 1]$ гарантира дека Леонид ќе купи барем еден сувенир. Доколку овие услови не се исполнети, Вашето решение ќе ја добие пресудата `Output isn't correct: Invalid argument`. Забележете дека спротивно на $P[0]$, вредноста на $P[N - 1]$ не е наведена во влезот.
- Процедурата може да се повика најмногу 5000 пати во секој тест случај.

Однесувањето на оценувачот е **неадаптивно**. Ова значи дека низата од цени P е фиксирана пред да се повика `buy_souvenirs`.

Ограничувања

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$ за секое i такво што $0 \leq i < N$.
- $P[i] > P[i + 1]$ за секое i такво што $0 \leq i < N - 1$.

Подзадачи

Подзадача	Поени	Дополнителни ограничувања
1	4	$N = 2$
2	3	$P[i] = N - i$ за секое i такво што $0 \leq i < N$.
3	14	$P[i] \leq P[i + 1] + 2$ за секое i такво што $0 \leq i < N - 1$.
4	18	$N = 3$
5	28	$P[i + 1] + P[i + 2] \leq P[i]$ за секое i такво што $0 \leq i < N - 2$. $P[i] \leq 2 \cdot P[i + 1]$ за секое i такво што $0 \leq i < N - 1$.
6	33	Без дополнителни ограничувања.

Пример

Да го разгледаме следниот повик.

```
buy_souvenirs(3, 4)
```

Постојат $N = 3$ типови на сувенири и $P[0] = 4$. Обрнете внимание дека постојат само три можни низи од цени P : $[4, 3, 2]$, $[4, 3, 1]$ и $[4, 2, 1]$.

Да претпоставиме дека `buy_souvenirs` ја повикува `transaction(2)`. Да претпоставиме дека повикот враќа $([2], 1)$, што означува дека Леонид купил еден сувенир од типот 2 и продавачот му вратил 1 паричка. Забележете дека ова ни овозможува да заклучиме дека $P = [4, 3, 1]$, бидејќи:

- За $P = [4, 3, 2]$, повикот `transaction(2)` би вратил $([2], 0)$.
- За $P = [4, 2, 1]$, повикот `transaction(2)` би вратил $([1], 0)$.

Потоа `buy_souvenirs` може да ја повика `transaction(3)`, која што враќа $([1], 0)$, што означува дека Леонид купил еден сувенир од типот 1 и продавачот му вратил 0 монети. Досега, вкупно, тој купил еден сувенир од типот 1 и еден сувенир од типот 2.

Конечно, `buy_souvenirs` може да ја повика `transaction(1)`, која што враќа $([2], 0)$, што означува дека Леонид купил еден сувенир од типот 2. Да забележиме дека тука можевме да го искористиме и повикот `transaction(2)`. Во овој момент, вкупно Леонид има еден сувенир од типот 1 и два сувенири од типот 2, според побарувањата.

Пример-оценувач

Формат на влез:

N

$P[0] \quad P[1] \quad \dots \quad P[N-1]$

Формат на излез:

$Q[0] \quad Q[1] \quad \dots \quad Q[N-1]$

Овде $Q[i]$ е вкупниот број на купени сувенири од тип i за секое i такво што $0 \leq i < N$.